// TECHNICAL BLUEPRINT · ENGINEERING REFERENCE

Implementation Blueprint

for the Workflow OS.

Platform layers · Module contracts · API boundaries · Deployment

This document is the engineering reference for the Esheria Legal Workflow OS. It maps the five-layer platform architecture, defines module responsibilities and connector contracts, specifies the canonical object model and workflow state machine, establishes the tenancy and auth model, and provides deployment topology guidance. Use after the Executive Proposal for implementation planning.

SECTIONS	ENTITIES	DIAGRAMS	LAYERS
8	9	8	L1--L5

DOCUMENT TYPE

Technical Architecture Blueprint

AUDIENCE

Engineering · CTO · Technical leads

COMPILE WITH

```
xelatex esheria-technical-blueprint.tex
```

01 System Architecture

// SEC 01

Five-layer stack — control plane, data plane, and experience surfaces

Architecture Overview

The platform is structured as five horizontal layers. Each layer has a bounded responsibility and communicates only with adjacent layers. The canonical architecture fact: **no layer has a direct dependency on a non-adjacent layer.**

Layer	Name	Canonical Prefix	Responsibility
L5	**Experience**	/ (frontend routing)	Institution Workspace, Operator Console, Public surface. Never calls `/internal/*`.
L4	**Control Plane**	`/api/workflows/*`	Workflow Engine, State Machine, Approval Router, Escalation Manager, Audit/Event Log.
L3	**Object Model**	`schema.*`	Nine canonical entities shared across all layers. Schema is locked in P1 before service code.
L2	**Connectors**	`/internal/*`	Connector Gateway mediating all platform-to-service calls. Owns retry, auth, error normalisation.
L1	**Existing Services**	`(service-native)`	Current repos: `esheria-ocr`, `contract-engine`, `esheria-ai`, `lexdraft`, `esheria-analytics`. Unchanged.

// CANONICAL ARCHITECTURE DECISION

The Canonical Object Model (L3) must be defined and locked in Phase 1 before any service code is written in Phase 2. Schema changes within a major version are treated as public API breaking changes and require migration plans, backward-compat periods, and explicit engineering approval.

Control Plane vs Data Plane

- **Control Plane (L4):** Governs workflow lifecycle — creates `WorkflowRun` objects, routes steps to operators, enforces SLAs, manages approval chains, and writes every state transition to the immutable audit log.
- **Data Plane (L2+L1):** Executes capability operations — OCR, extraction, compliance check, research, drafting. Reads and writes canonical objects at L3. Never controls workflow routing decisions.
- **Boundary rule:** A capability service *never* triggers workflow transitions. It returns results to the Control Plane, which decides the next step.

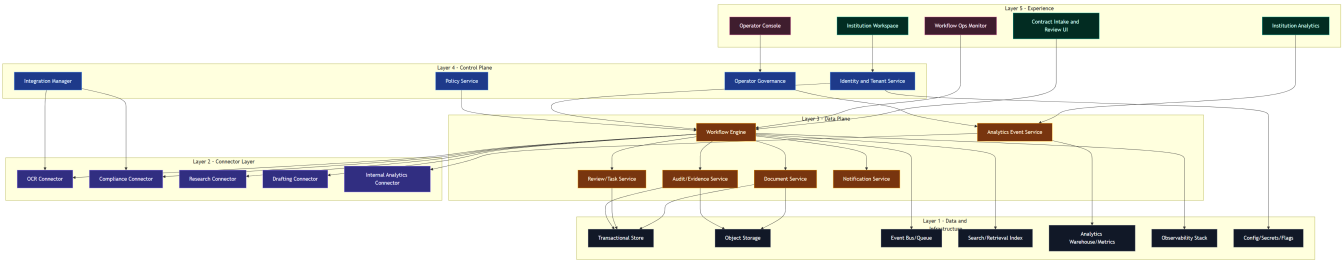

Figure 1: *

Figure 01 — Five-layer platform architecture with control/data plane boundary

---

02 API Boundaries

// SEC 02

External /api/ vs internal /internal/* — no cross-contamination*

Boundary Contract

Two API namespaces exist. They must never be conflated:

Prefix	Caller	Auth	Purpose
`/api/*`	**L5 frontends, external clients**	JWT (org-scoped)	All customer and operator product actions. Versioned, documented.
`/internal/*`	**L2 Connector Gateway only**	Service-to-service token	Platform-to-capability service calls. Never exposed externally.

External API Catalog

Endpoint Group	Method(s)	Description
`/api/auth/token`	POST	Exchange credentials for org-scoped JWT
`/api/auth/refresh`	POST	Refresh access token
`/api/orgs/{id}`	GET, PATCH	Org profile and policy set
`/api/matters`	GET, POST	List and create legal matters
`/api/matters/{id}`	GET, PATCH, DELETE	Matter detail and lifecycle
`/api/documents`	GET, POST	Upload and list documents
`/api/documents/{id}`	GET, DELETE	Document detail and deletion
`/api/workflows`	GET, POST	List and trigger workflow runs
`/api/workflows/{id}`	GET	WorkflowRun state + step history
`/api/reviews`	GET, POST	List and submit human reviews
`/api/tasks`	GET, PATCH	Assignee-scoped task queue
`/api/analytics/summary`	GET	Org-level workflow metrics
`/api/analytics/sla`	GET	SLA compliance by matter type
`/api/integrations`	GET, POST	DMS/CRM/ERP connector config

Internal Connector Endpoints

Internal Endpoint	Target Service	Contract
`/internal/ocr/parse`	**esheria-ocr**	Submit PDF/image, receive parsed text + metadata
`/internal/ocr/classify`	**esheria-ocr**	Classify document type from parsed content
`/internal/contracts/extract`	**contract-engine**	Extract clauses, parties, obligations from text
`/internal/contracts/risk`	**contract-engine**	Score risk per clause against policy set
`/internal/research/search`	**esheria-ai**	Semantic search over precedent corpus
`/internal/research/cite`	**esheria-ai**	Retrieve citations for a given risk or clause
`/internal/drafting/suggest`	**lexdraft**	Generate fallback language for risky clauses
`/internal/drafting/redline`	**lexdraft**	Produce tracked-change redline document
`/internal/analytics/emit`	**esheria-analytics**	Emit workflow event to analytics pipeline
`/internal/analytics/query`	**esheria-analytics**	Query institution metrics (internal only)

> // BOUNDARY ENFORCEMENT
>
> The Connector Gateway is the *only* component permitted to call `/internal/*` endpoints. No frontend, no workflow operator, and no other backend service may bypass it. This is the primary isolation mechanism between the platform and capability services.

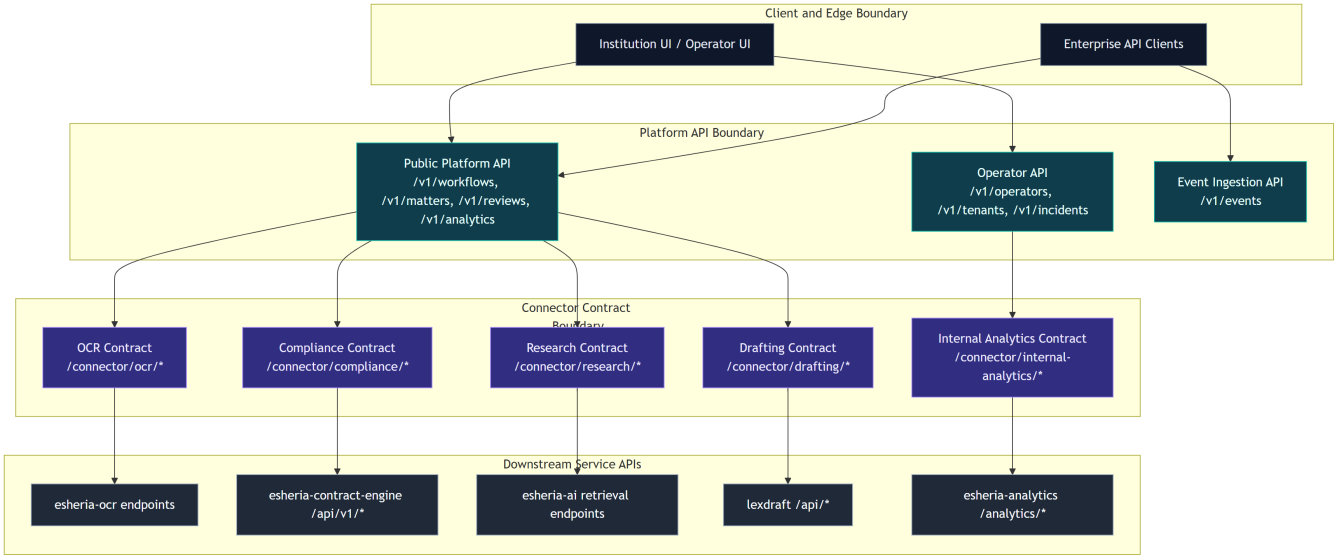

Figure 2: *

Figure 02 — Platform API and connector boundary map

03 Canonical Ontology Model

// SEC 03

Nine entities — the shared data contract for the entire platform

Entity Registry

All platform services operate on the same nine canonical entities. No service maintains private copies of these objects — they are the authoritative state of the platform.

Entity	ID	Description	Key Fields
`Organization`	E1	Root tenant. Owns all nested objects.	`org_id, name, sector, policy_set_id, jurisdiction`
`User`	E2	Identity scoped to an Org.	`user_id, org_id, email, role, permissions`
`Matter`	E3	Legal business context (case, contract, compliance review).	`matter_id, org_id, matter_type, status, owner`
`Document`	E4	Uploaded file with OCR status and version chain.	`doc_id, matter_id, source_type, ocr_status, version`
`Clause`	E5	Extracted clause attached to a Document.	`clause_id, doc_id, clause_type, text, risk_ids`
`Risk`	E6	Compliance finding linked to a Clause.	`risk_id, clause_id, severity, policy_violated, remediation`
`WorkflowRun`	E7	Single execution of a named workflow template.	`run_id, matter_id, workflow_type, current_step, status`
`Action`	E8	Atomic step record within a WorkflowRun.	`action_id, run_id, actor_id, type, input, output (jsonb)`
`Review`	E9	Human reviewer decision on a Document.	`review_id, doc_id, reviewer_id, findings (jsonb), decision`

Relationship Rules

- Every entity except `Organization` carries an `org_id` foreign key. Row-level security enforces org-scoped isolation at the database layer.
- `Matter` is the primary grouping unit. All documents, workflow runs, and reviews attach to a Matter.
- `Clause` and `Risk` are write-once after extraction. Risk findings are appended, never mutated — only resolved through a `Review` decision.
- `Action` records are immutable once written. They form the audit trail for a `WorkflowRun`.
- A `Citation` (not listed above as a top-level entity) is an evidence record linking an `Action` to a source document — used by the Research and Compliance operators.

> `// SCHEMA LOCK POLICY`
>
> The entity schema is treated as a public API contract. All field additions require a non-breaking migration. Field removals or type changes require a major version increment, a migration plan, and a deprecation window of at minimum one full sprint cycle before enforcement.

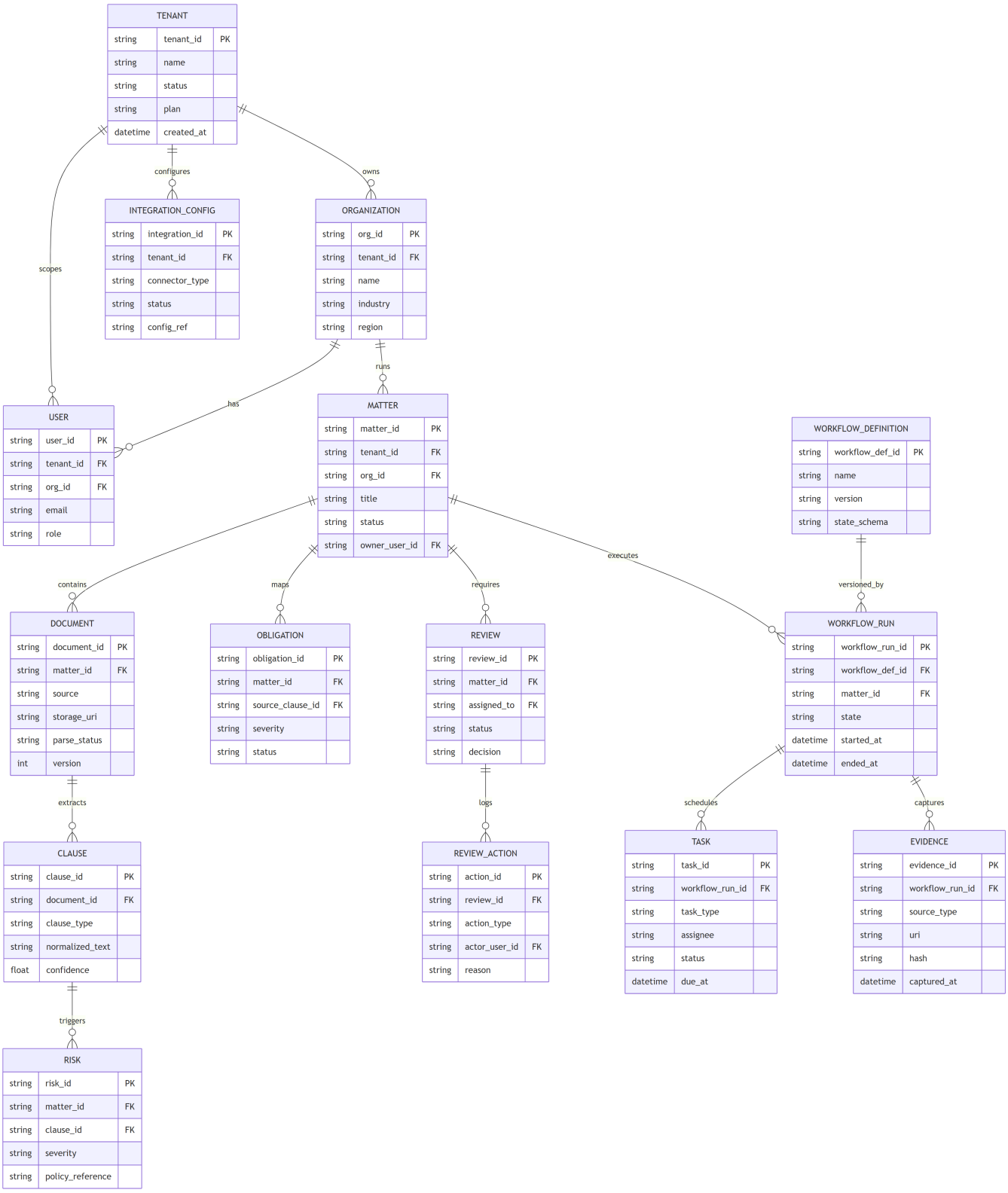

Figure 3: *

Figure 03 — Canonical entity-relationship diagram for the nine platform objects

04 Workflow Engine Design

// SEC 04

State machine runtime with deterministic transitions and full audit

Design Principles

- **Deterministic transitions:** Every step has a single defined next-state for each possible outcome. No implicit routing logic.
- **Idempotent operators:** Each operator step can be safely retried without side effects — output objects are upserted by `(run_id, step_id)`.
- **Immutable audit log:** Every state transition writes an append-only record to the event log. Log records are never updated or deleted.
- **Human-in-loop:** The Approval Router is a first-class step, not an afterthought. Workflows pause in `AWAITING_REVIEW` until a human decision is received.
- **SLA enforcement:** The Escalation Manager polls pending reviews and fires escalation events at configured deadline thresholds (typically 80% and 100% of SLA window).

WorkflowRun Lifecycle States

State	Description	Valid Transitions
`PENDING`	Run created, not yet started	`-> RUNNING`
`RUNNING`	Active operator step in execution	`-> AWAITING_REVIEW, FAILED`
`AWAITING_REVIEW`	Paused pending human reviewer decision	`-> RUNNING, ESCALATED`
`ESCALATED`	SLA threshold exceeded, escalation fired	`-> AWAITING_REVIEW`
`APPROVED`	Reviewer accepted; routing to close	`-> COMPLETED`
`REJECTED`	Reviewer rejected; routing to remediate	`-> RUNNING` (remediation step)
`COMPLETED`	All steps done, analytics emitted	(terminal)
`FAILED`	Operator error beyond retry budget	`-> RUNNING` (manual restart only)
`CANCELLED`	Explicitly terminated by operator	(terminal)

Operator Registry — Contract Intake & Review

Operator	Type	Input	Output Object
`IngestionOperator`	AI	`doc_id`	`DocumentVersion`, parsed text
`ExtractionOperator`	AI	`doc_id`, parsed text	`Clause[]` attached to `doc_id`
`ComplianceOperator`	AI	`Clause[]`, `policy_set_id`	`Risk[]` with severity scores
`ResearchOperator`	AI	`Risk[]`, `Clause[]`	`Citation[]` linked to findings
`DraftingOperator`	AI	`Risk[]`, `Clause[]`	Draft suggestions per clause
`ReviewOperator`	AI	All prior outputs	`Review` briefing record
`HumanReviewer`	Human	`Review` briefing	Decision (`APPROVED` / `REJECTED`)
`ApprovalRouter`	System	`Review` decision	Routes to `APPROVED` or `REJECTED` state

> // OPERATOR ISOLATION
>
> Operators must not communicate with each other directly. All inter-step data flows through the canonical object model (L3). An operator reads its input from the `Action.input` record created by the Control Plane, executes via its connector, and writes results back as the `Action.output` jsonb field.

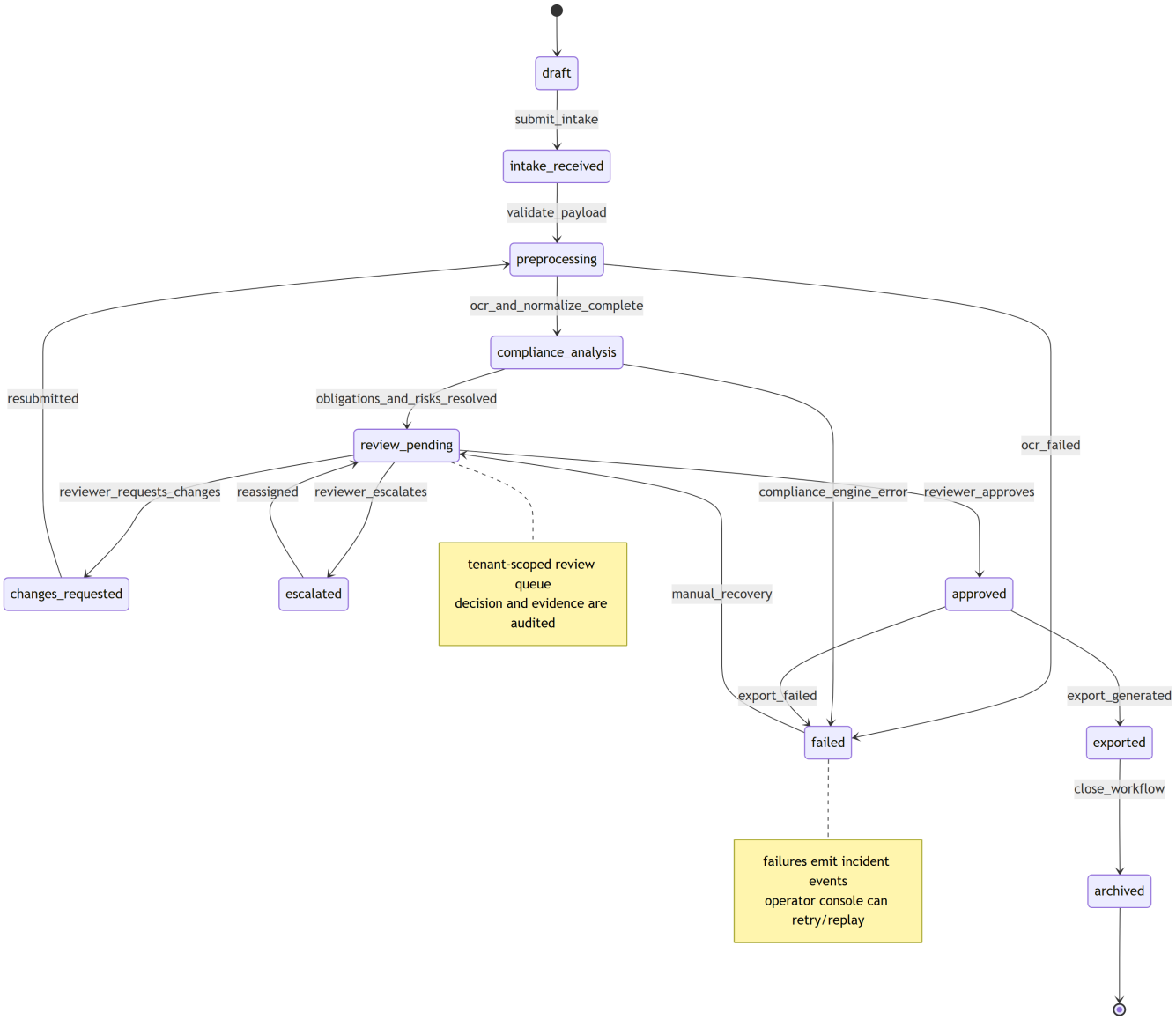

Figure 4: *

Figure 04 — Workflow state machine with deterministic transitions and operator registry

05 Tenancy & Auth Model

// SEC 05

Multi-tenant with org-scoped JWT, RBAC, and analytics domain separation

Tenancy Architecture

The platform uses a **logical multi-tenancy model** with shared infrastructure and database-layer isolation. Org-scoped row-level security (RLS) on PostgreSQL enforces data isolation at the query layer — no application-level filtering is trusted as the primary isolation mechanism.

JWT Structure

Every authenticated request carries a JWT containing:

```
{ "sub": "<user_id>", "org": "<org_id>", "role": "reviewer | admin | api_client | operator | super_admin | auditor", "scopes": ["matters:read", "documents:write", "workflows:trigger", ...], "exp": <unix_timestamp>, "iss": "esheria-auth" }
```

Role Definitions

Role	Surface	Capabilities
`reviewer`	Institution	Read matters, documents. Submit review decisions. View own analytics.
`admin`	Institution	Full org management. Configure workflows, users, integrations, policy sets.
`api_client`	Institution	Machine-to-machine. Scoped to explicit permission grants only.
`operator`	Esheria internal	Monitor workflow runs. Access operator console. View all org analytics.
`super_admin`	Esheria internal	Full platform control. Maintenance mode, service drain, force-advance.
`auditor`	Esheria internal	Read-only access to all audit logs and event streams.

> // OPERATOR CONSOLE ISOLATION
>
> The Operator Console is a completely separate auth flow from the Institution Workspace. Institution users cannot access it under any circumstance — not via role elevation, not via URL manipulation. The Operator Console is served from a separate subdomain with a separate JWT issuer. All operator impersonation actions create immutable audit records with a mandatory justification field.

Analytics Domain Separation

Two analytics domains exist and must remain architecturally isolated:

- **Institution Analytics:** Customer-facing. Scoped to a single `org_id`. Exposed via `/api/analytics/*`. Shows workflow throughput, SLA compliance, risk trends, and reviewer performance for that org only.
- **Esheria Internal Analytics:** Operator-facing. Cross-org. Never accessible to customers. Exposed only inside the Operator Console. Shows platform health, service latency, error rates, and cross-org usage patterns.

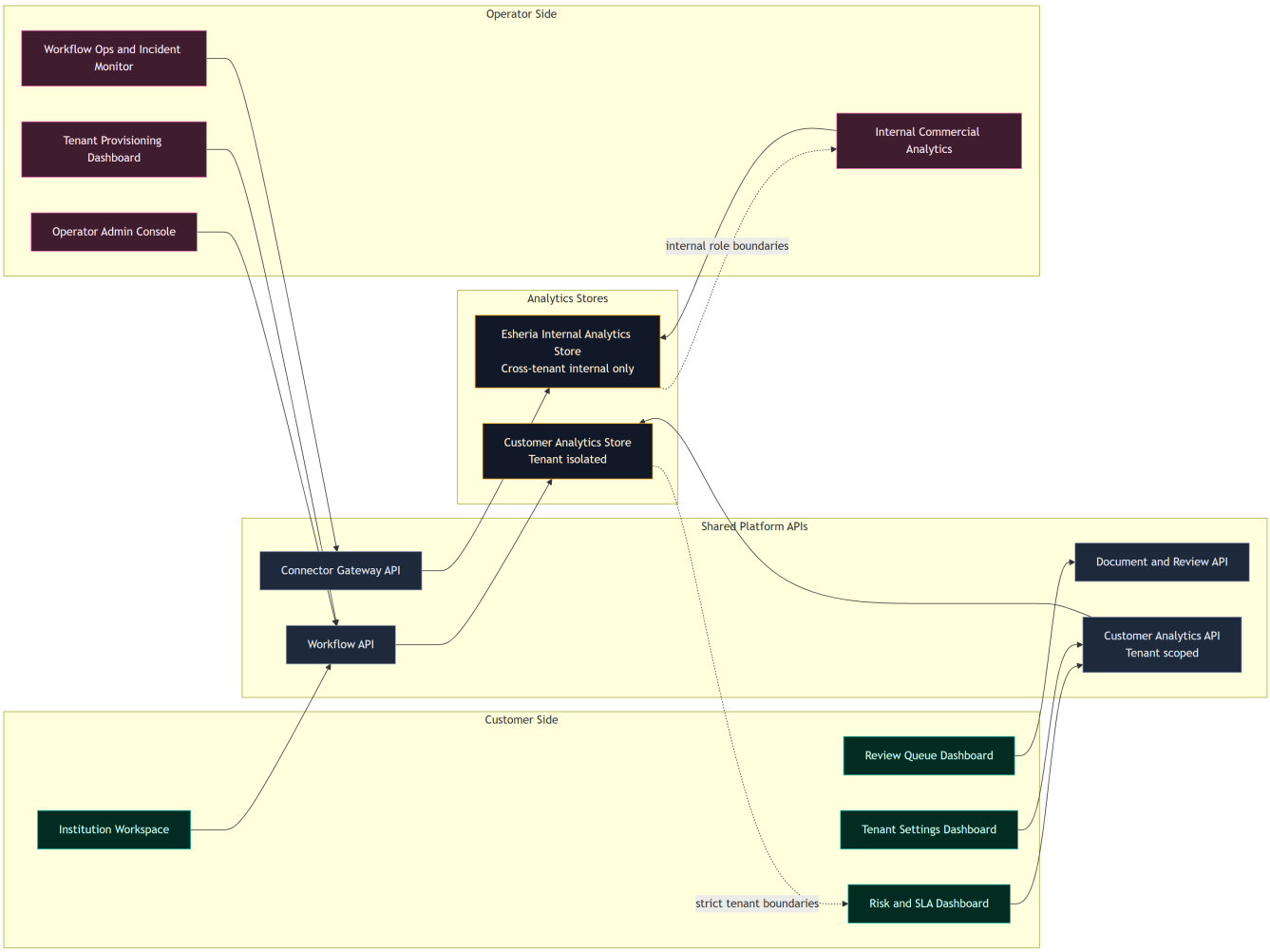

Figure 5: *

Figure 05 — Tenant-scoped institution analytics separated from Esheria internal analytics

06 Control vs Data Plane

// SEC 06

Responsibility split — orchestration above, execution below

Plane Responsibilities

Concern	Control Plane (L4)	Data Plane (L2 + L1)
Primary role	Orchestration and governance	Capability execution
State ownership	`WorkflowRun` lifecycle, step routing	`Action.output`, entity writes
Routing decisions	Yes — next step, branching, escalation	No — never controls routing
Audit trail	Writes every state transition to event log	Writes `Action` records only
Auth model	Validates org-scoped JWT on every request	Service-to-service token from Gateway
SLA enforcement	Escalation Manager fires threshold events	Not applicable
Human interaction	Routes to Approval Router, parks run	Not applicable
Retry logic	Sets retry budget per step type	Connector implements retry with backoff

Key Boundary Rules

CONSTRAINT A capability operator (L1/L2) must never trigger a `WorkflowRun` state transition directly. It returns a result and the Control Plane decides next-state.

CONSTRAINT The Connector Gateway (L2) must never hold business logic. It is a routing and normalisation layer only — auth, retry, circuit-break, error-shape.

DECISION Emergency platform controls (maintenance mode, service drain, force-advance workflow) require a 4-eyes confirmation inside the Operator Console. Single-operator execution is blocked at the API level.

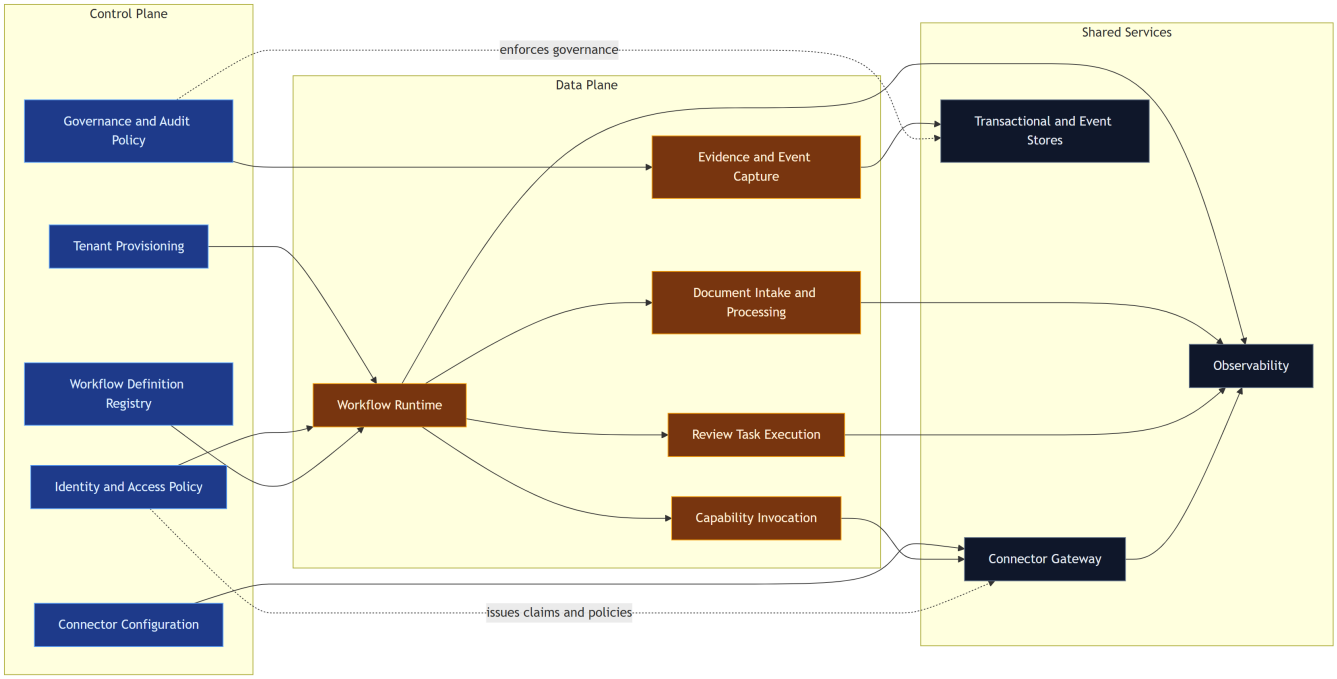

Figure 6: *

Figure 06 — Control-plane and data-plane responsibility split with boundary enforcement rules

07 Deployment Topology

// SEC 07

Recommended MVP infrastructure stack and service boundaries

Infrastructure Components

Component	Technology	Responsibility
API Gateway	`nginx / AWS ALB`	TLS termination, rate limiting, header validation
Auth Service	`custom JWT issuer`	Token generation, refresh, revocation
Platform Backend	`Node.js / Go`	Workflow Engine, Control Plane, Connector Gateway
Message Queue	`Redis Streams`	Async step execution, decouples operators from engine
Primary Database	`PostgreSQL + RLS`	Canonical object persistence with org-scoped isolation
Document Store	`S3-compatible`	Raw files, OCR output, redline documents
Vector Store	`pgvector / Pinecone`	Research operator semantic search index
Cache	`Redis`	Session tokens, workflow step debounce
Analytics Pipeline	`ClickHouse / Kafka`	Append-only event stream for institution analytics
Monitoring	`Prometheus + Grafana`	Service health, latency, error rate dashboards

Request Flow (Contract Intake)

```
Client -> API Gateway (TLS + rate limit) -> Auth Service (JWT validate) -> Platform Backend /api/documents
(POST) -> PostgreSQL (create Document record) -> Redis Streams (emit step:ingestion event) -> Platform Backend
(consume event) -> Connector Gateway /internal/ocr/parse -> esheria-ocr (parse + classify) -> PostgreSQL (write
DocumentVersion) -> Redis Streams (emit step:extraction event) -> ... (continues per operator)
```

Out-of-Scope for MVP Phase 2

- Full enterprise SSO / SAML integration (post-P3)
- Multi-region active-active deployment (post-P4)
- Universal DMS/CRM/ERP connector suite (post-P4 — one reference integration only in P4)
- Auth system unification across existing repos (each repo retains its own auth in P2)
- All nine workflow types — only Contract Intake & Review in P2
- Bi-directional analytics (institution can read, not write, in P2)

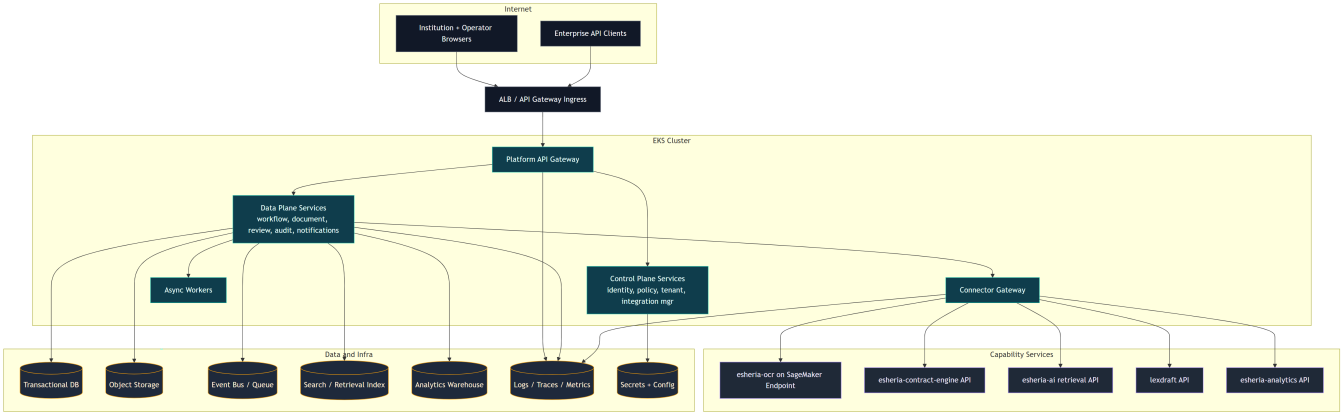

Figure 7: *

Figure 07 — Deployment topology for MVP: API gateway, backend, queues, and data stores

08 Integration Architecture

// SEC 08

Connector Gateway contracts and service integration map

Connector Contract Specification

Each connector in L2 must implement the following interface contract:

```
interface Connector  id: string; // e.g. "ocr-connector" targetService: string; // e.g. "esheria-ocr" status:
"active" | "planned" | "future";
//  Called  by  Connector  Gateway  for  every  request  call(endpoint:   string,  payload:   object):
Promise<ConnectorResult>;
// Error normalisation — maps service errors to platform error codes normalise(error: ServiceError): Plat-
formError;
// Health check — called by monitoring every 30s health(): Promise< ok: boolean; latencyMs: number >;
interface ConnectorResult  success: boolean; data: object; metadata:  serviceVersion: string; latencyMs:
number ;
```

Connector Registry

Connector ID	Target Service	MVP Status	Notes
`ocr-connector`	`esheria-ocr`	Active	P2 priority. First connector to build.
`contract-connector`	`contract-engine`	Active	P2 priority. Extraction + risk scoring.
`research-connector`	`esheria-ai`	Active	P2 priority. Semantic search + citations.
`drafting-connector`	`lexdraft`	Planned	P3. Drafting can use stub in P2.
`analytics-connector`	`esheria-analytics`	Planned	P3. Analytics emit stubbed in P2.
`lexchat-connector`	`lexchat-plugin`	Future	Post-P4. Experimental surface only.

Gateway Behaviour

- **Retry policy:** Exponential backoff, max 3 retries, 500ms base delay. `4xx` errors are *not* retried (client error). `5xx` and timeouts are retried.
- **Circuit breaker:** Opens after 5 consecutive failures within a 30-second window. Half-open probe every 10s. Failing-open is never permitted — the workflow step fails cleanly.
- **Timeout budget:** Per-connector timeout configured in registry. Default 10s. OCR has 60s budget. Research has 30s budget.
- **Error normalisation:** Every connector maps its service's error taxonomy to the platform's 6-class error model: `AUTH_ERROR`, `VALIDATION_ERROR`, `SERVICE_UNAVAILABLE`, `RATE_LIMITED`, `TIMEOUT`, `INTERNAL_ERROR`.

> // CONNECTOR PRIORITY ORDER — P2
>
> Build in this order: `ocr-connector` first (simplest, no ML dependency, validates gateway plumbing), then `contract-connector` (highest business value in MVP), then `research-connector` (needed for citation evidence in Review briefing). Stub `drafting-connector` and `analytics-connector` in P2 to unblock demo.

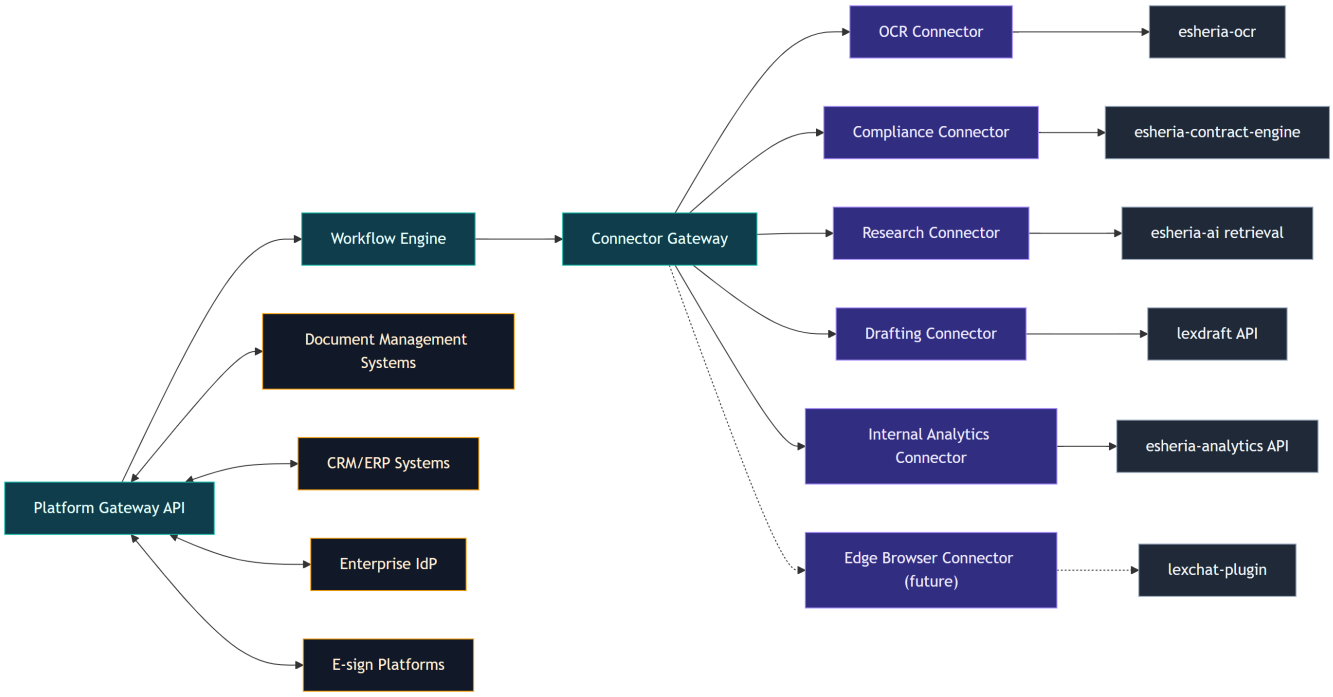

Figure 8: *

Figure 08 — Connector gateway integration architecture across all five capability services

// IMPLEMENTATION GUIDANCE

Phase 2 Build Priorities

Priority Order

1. **Schema definition (P1 exit gate):** Define all nine canonical entities in a PostgreSQL migration. Apply RLS policies. Review and lock with all repo owners before writing any application code.
2. **Workflow Engine core:** `WorkflowRun` CRUD, state machine transitions, event log writes. No connector calls yet — use stubs.
3. **Connector Gateway skeleton:** Auth passthrough, retry/backoff, error normalisation. Implement against stubs before wiring real services.
4. **OCR connector:** First real connector. Validates the gateway contract end-to-end.
5. **Contract connector:** Extraction and risk scoring. Unlocks the core value step.
6. **Research connector:** Citation retrieval. Completes the evidence chain.
7. **Reviewer dashboard v1:** Minimal UI — matter list, document viewer, risk findings, approve/reject action. Authentication via institution workspace auth flow.
8. **End-to-end demo flow:** Single contract through all 10 steps. Audit log visible. Human reviewer makes a decision. Analytics event emitted (stubbed).

Explicit Out-of-Scope for P2

- Full enterprise SSO, SAML, or external IdP integration
- Auth unification across existing repos (each keeps its own auth in P2)
- Universal DMS/CRM/ERP integration suite (one reference only in P4)
- All nine workflow type templates (Contract Intake only in P2)

- Bi-directional analytics (institution read-only in P2; full dashboard in P3)
- Production multi-region or disaster recovery configuration
- Drafting and analytics connectors (stub in P2, build in P3)

> // SCHEMA FIRST — ALWAYS
>
> The single most common cause of platform architecture failures is writing service code before agreeing on the canonical data model. In P1, the schema review meeting is the most important engineering meeting. No P2 code is written before the schema is signed off by all repo owners.

Deep Evidence

- `docs/discovery/repo-deep-scan.md` — full assessment of all five existing repos
- `docs/discovery/api-catalog.md` — current endpoint inventory per service
- `docs/architecture/consistency-report.md` — 7 resolved canonical conflicts, 11 confirmed decisions
- Executive Proposal — `esheria-executive-proposal.tex` — strategic context for this blueprint